

PrO-4. Execute data preprocessing tasks on **the given** dataset using Python libraries.

Dataset 1: Student Performance Dataset

1. Import the Student Performance dataset using the Pandas library.

```
import pandas as pd          URK25CS7091

df = pd.read_csv("Student_Performance_Dataset.csv")

print(df)
```

30	S031	Male	20	77.0	2.6	66.0
31	S032	Female	18	92.0	4.6	87.0
32	S033	Male	21	74.0	2.3	63.0
33	S034	Female	20	89.0	4.1	84.0
34	S035	Male	19	81.0	3.0	73.0
35	S036	Female	18	93.0	5.1	93.0
36	S037	Male	20	70.0	1.9	57.0
37	S038	Female	21	87.0	4.3	82.0
38	S039	Male	22	79.0	2.9	71.0
39	S040	Female	19	90.0	4.7	88.0
40	S005	Male	21	81.0	3.0	72.0
41	S041	Male	23	100.0	10.0	100.0
42	S042	Female	17	45.0	0.5	20.0

2. Examine the dataset and identify missing values, duplicate records, and inconsistent data types.

```
print("Data Types:")
print(df.dtypes)
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
print("\nDuplicate Records:")
print(df.duplicated().sum())
```

```
Data Types:
Student_ID      object
Gender          object
Age             int64
Attendance      float64
Study_Hours     float64
Assignment_Score float64
Internal_Marks  float64
Final_Marks     int64
dtype: object
```

```
Missing Values:
Student_ID      0
Gender          1
Age             0
Attendance      1
Study_Hours     1
Assignment_Score 1
Internal_Marks  1
Final_Marks     0
dtype: int64
```

```
Duplicate Records:
1
```

3. Handle missing values using suitable preprocessing techniques such as mean, median, or mode imputation.

```
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

```
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].median())
df['Study_Hours'] = df['Study_Hours'].fillna(df['Study_Hours'].median())
df['Assignment_Score'] = df['Assignment_Score'].fillna(df['Assignment_Score'].median())
df['Internal_Marks'] = df['Internal_Marks'].fillna(df['Internal_Marks'].median())
```

```
print(df.isnull().sum())
```

```
URK25CS7091
```

```
Student_ID      0
Gender          0
Age             0
Attendance      0
Study_Hours     0
Assignment_Score 0
Internal_Marks  0
Final_Marks     0
dtype: int64
```

4. Encode the categorical attribute **Gender** into numerical values using Label Encoding or One-Hot Encoding.

```
from sklearn.preprocessing import LabelEncoder  
  
le = LabelEncoder()  
  
df['Gender'] = le.fit_transform(df['Gender'])  
  
print(df[['Gender']])
```

URK25CS7091|

	Gender
0	1
1	0
2	1
3	0
4	1
5	0
6	1
7	0
8	1
9	0
10	1
11	0
12	1
13	0
14	1
15	0
16	1
17	0
18	1
19	0
20	1
21	0
22	1
23	0
24	1
25	0
26	1
27	1
28	1
29	0
30	1
31	0
32	1
33	0
34	1
35	0
36	1
37	0
38	1
39	0
40	1
41	1
42	0

5. Detect outliers in the **Study_Hours** and **Final_Marks** attributes using the IQR or Box Plot method and treat them appropriately.

```

for column in ['Study_Hours', 'Final_Marks']:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    print(column, "Outliers:")
    print(df[(df[column] < lower) | (df[column] > upper)])

    df[column] = df[column].clip(lower, upper)

```

URK25CS7091

Study_Hours Outliers:

	Student_ID	Gender	Age	Attendance	Study_Hours	Assignment_Score	\
41	S041	1	23	100.0	10.0	100.0	

	Internal_Marks	Final_Marks
41	30.0	100

Final_Marks Outliers:

	Student_ID	Gender	Age	Attendance	Study_Hours	Assignment_Score	\
42	S042	0	17	45.0	0.5	20.0	

	Internal_Marks	Final_Marks
42	8.0	25

6. Normalize the numerical attributes using Min-Max Scaling.

```

from sklearn.preprocessing import MinMaxScaler

columns = ['Age', 'Attendance', 'Study_Hours',
           'Assignment_Score', 'Internal_Marks', 'Final_Marks']

scaler = MinMaxScaler()

df[columns] = scaler.fit_transform(df[columns])

```

```

print(df)

```

URK25CS7091						
30	S031	1	0.500000	0.581818	0.316981	0.57500
31	S032	0	0.166667	0.854545	0.618868	0.83750
32	S033	1	0.666667	0.527273	0.271698	0.53750
33	S034	0	0.500000	0.800000	0.543396	0.80000
34	S035	1	0.333333	0.654545	0.377358	0.66250
35	S036	0	0.166667	0.872727	0.694340	0.91250
36	S037	1	0.500000	0.454545	0.211321	0.46250
37	S038	0	0.666667	0.763636	0.573585	0.77500
38	S039	1	0.833333	0.618182	0.362264	0.63750
39	S040	0	0.333333	0.818182	0.633962	0.85000
40	S005	1	0.666667	0.654545	0.377358	0.65000
41	S041	1	1.000000	1.000000	1.000000	1.00000
42	S042	0	0.000000	0.000000	0.000000	0.00000

	Internal_Marks	Final_Marks
0	0.727273	0.665116
1	0.818182	0.720930
2	0.545455	0.404651
3	0.909091	0.888372
4	0.636364	0.534884
5	0.863636	0.776744
6	0.454545	0.218605
7	0.863636	0.813953
8	0.500000	0.311628
9	0.954545	0.944186
10	0.681818	0.609302
11	0.909091	0.832558
12	0.590909	0.441860

13	0.954545	0.906977
14	0.727273	0.627907
15	0.772727	0.702326
16	0.454545	0.255814
17	0.954545	0.925581
18	0.681818	0.572093
19	0.909091	0.851163
20	0.590909	0.479070
21	0.772727	0.683721
22	0.545455	0.386047
23	1.000000	0.962791
24	0.681818	0.590698
25	0.818182	0.739535
26	0.636364	0.516279
27	0.863636	0.795349
28	0.590909	0.497674
29	0.727273	0.665116
30	0.545455	0.423256
31	0.909091	0.813953
32	0.500000	0.367442
33	0.818182	0.758140
34	0.636364	0.553488
35	0.954545	0.925581
36	0.454545	0.274419
37	0.818182	0.720930
38	0.636364	0.516279
39	0.909091	0.832558
40	0.636364	0.534884
41	1.000000	1.000000
42	0.000000	0.000000

- Standardize the numerical attributes using StandardScaler and compare the transformed values with normalized data.

```

from sklearn.preprocessing import StandardScaler

standard = StandardScaler()

df_standard = df.copy()

df_standard[colums] = standard.fit_transform(df_standard[colums])

print("Normalized Data:")
print(df[colums].head())

print("\nStandardized Data:")
print(df_standard[colums].head())

```

URK25CS7091

Normalized Data:

	Age	Attendance	Study_Hours	Assignment_Score	Internal_Marks	\
0	0.166667	0.854545	0.452830	0.7250	0.727273	
1	0.333333	0.781818	0.528302	0.7750	0.818182	
2	0.500000	0.563636	0.301887	0.5625	0.545455	
3	0.166667	0.909091	0.679245	0.8750	0.909091	
4	0.666667	0.654545	0.377358	0.6500	0.636364	

Final_Marks

0	0.665116
1	0.720930
2	0.404651
3	0.888372
4	0.534884

Standardized Data:

	Age	Attendance	Study_Hours	Assignment_Score	Internal_Marks	\
0	-1.169130	0.822629	-0.078246	0.098119	0.037557	
1	-0.461065	0.416625	0.319927	0.381755	0.498977	
2	0.246999	-0.801384	-0.874592	-0.823701	-0.885282	
3	-1.169130	1.127131	1.116273	0.949029	0.960397	
4	0.955064	-0.293880	-0.476419	-0.327337	-0.423862	

Final_Marks

0	0.158257
1	0.402019
2	-0.979301
3	1.133306
4	-0.410522

8. Save the cleaned and preprocessed dataset as **student_preprocessed.csv**.

```

df.to_csv("student_preprocessed.csv", index=False)

print("Dataset saved as student_preprocessed.csv")

```

Dataset saved as student_preprocessed.csv

Dataset 2: House Price Prediction Dataset

1. Import the House Price dataset into a Pandas DataFrame.

```
import pandas as pd
```

```
df = pd.read_csv("Housing.csv")
```

```
print(df)
```

```
URK25CS7091|
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	
..	...	...	...	...	...	...	...	...	
540	1820000	3000	2	1	1	yes	no	yes	
541	1767150	2400	3	1	1	no	no	no	
542	1750000	3620	2	1	1	yes	no	no	
543	1750000	2910	3	1	1	no	no	no	
544	1750000	3850	3	1	2	yes	no	no	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished
..	...	...	...	...	...
540	no	no	2	no	unfurnished
541	no	no	0	no	semi-furnished
542	no	no	0	no	unfurnished
543	no	no	0	no	furnished
544	no	no	0	no	unfurnished

```
[545 rows x 13 columns]
```

2. Identify missing values and replace them using appropriate statistical measures.

```

print("Missing values before:")
print(df.isnull().sum())

for column in df.select_dtypes(include='number').columns:
    df[column] = df[column].fillna(df[column].median())

for column in df.select_dtypes(include='object').columns:
    df[column] = df[column].fillna(df[column].mode()[0])

print("\nMissing values after:")
print(df.isnull().sum())          URK25CS7091

```

```

Missing values before:
price          0
area           0
bedrooms       0
bathrooms      0
stories        0
mainroad       0
guestroom      0
basement       0
hotwaterheating 0
airconditioning 0
parking        0
prefarea       0
furnishingstatus 0
dtype: int64

```

```

Missing values after:
price          0
area           0
bedrooms       0
bathrooms      0
stories        0
mainroad       0
guestroom      0
basement       0
hotwaterheating 0
airconditioning 0
parking        0
prefarea       0
furnishingstatus 0
dtype: int64

```

3. Remove duplicate records, if any, from the dataset.

```

print("Duplicates before:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates after:", df.duplicated().sum())    URK25CS7091

```

```

Duplicates before: 0
Duplicates after: 0

```

- Convert the categorical attributes **airconditioning** and **furnishingstatus** into numerical form using encoding techniques.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['airconditioning'] = le.fit_transform(df['airconditioning'])

df = pd.get_dummies(df, columns=['furnishingstatus'], dtype=int)

print(df.head())
```

```
URK25CS7091
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	\
0	no		1	2	yes
1	no		1	3	no
2	no		0	2	yes
3	no		1	3	yes
4	no		1	2	no

	furnishingstatus_furnished	furnishingstatus_semi-furnished	\
0	1	0	
1	1	0	
2	0	1	
3	1	0	
4	1	0	

	furnishingstatus_unfurnished
0	0
1	0
2	0
3	0
4	0

- Detect and remove outliers from the **Area** and **Price** columns using the IQR method.

```

for column in ['area', 'price']:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[column] >= lower) & (df[column] <= upper)]

print(df)

```

URK25CS7091

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
17	8960000	8500	3	2	4	yes	no	no	
18	8890000	4600	3	2	2	yes	yes	no	
19	8855000	6420	3	2	2	yes	no	no	
20	8750000	4320	3	1	2	yes	no	yes	
21	8680000	7155	3	2	1	yes	yes	yes	
..	...	...	...	...	...	...	...	...	
540	1820000	3000	2	1	1	yes	no	yes	
541	1767150	2400	3	1	1	no	no	no	
542	1750000	3620	2	1	1	yes	no	no	
543	1750000	2910	3	1	1	no	no	no	
544	1750000	3850	3	1	2	yes	no	no	

	hotwaterheating	airconditioning	parking	prefarea	\
17	no	1	2	no	
18	no	1	2	no	
19	no	1	1	yes	
20	yes	0	2	no	
21	no	1	2	no	
..	...	...	...	...	
540	no	0	2	no	
541	no	0	0	no	
542	no	0	0	no	
543	no	0	0	no	
544	no	0	0	no	

```

furnishingstatus_furnished  furnishingstatus_semi-furnished  \
17                          1                          0
18                          1                          0
19                          0                          1
20                          0                          1
21                          0                          0
..                          ...                          ...
540                         0                          0
541                         0                          1
542                         0                          0
543                         1                          0
544                         0                          0

furnishingstatus_unfurnished
17                          0
18                          0
19                          0
20                          0
21                          1
..                          ...
540                         1
541                         0
542                         1
543                         0
544                         1

[518 rows x 15 columns]

```

6. Perform feature scaling on **Area**, **Bedrooms**, **Bathrooms**, and **Price** using Min-Max Normalization.

```

from sklearn.preprocessing import MinMaxScaler

columns = ['area', 'bedrooms', 'bathrooms', 'price']

scaler = MinMaxScaler()

df[columns] = scaler.fit_transform(df[columns])

print(df[columns].head())

```

URK25CS7091

	area	bedrooms	bathrooms	price
17	0.774011	0.4	0.5	1.000000
18	0.333333	0.4	0.5	0.990291
19	0.538983	0.4	0.5	0.985437
20	0.301695	0.4	0.0	0.970874
21	0.622034	0.4	0.5	0.961165

7. Display summary statistics before and after preprocessing to compare the dataset quality.

```
print("Summary Statistics After Preprocessing:")
print(df.describe())
```

```
URK25CS7091
```

```
Summary Statistics After Preprocessing:
```

	price	area	bedrooms	bathrooms	stories \
count	518.000000	518.000000	518.000000	518.000000	518.000000
mean	0.389641	0.369170	0.387645	0.131274	1.791506
std	0.218033	0.206294	0.146136	0.233025	0.865727
min	0.000000	0.000000	0.000000	0.000000	1.000000
25%	0.229126	0.211299	0.200000	0.000000	1.000000
50%	0.347087	0.322034	0.400000	0.000000	2.000000
75%	0.527913	0.498305	0.400000	0.375000	2.000000
max	1.000000	1.000000	1.000000	1.000000	4.000000

	airconditioning	parking	furnishingstatus_furnished \
count	518.000000	518.000000	518.000000
mean	0.306950	0.652510	0.247104
std	0.461674	0.840756	0.431745
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	1.000000	1.000000	0.000000
max	1.000000	3.000000	1.000000

	furnishingstatus_semi-furnished	furnishingstatus_unfurnished
count	518.000000	518.000000
mean	0.416988	0.335907
std	0.493538	0.472763
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	1.000000	1.000000
max	1.000000	1.000000

8. Export the processed dataset to a CSV file named **houseprice_cleaned.csv**.

```
df.to_csv("houseprice_cleaned.csv", index=False)

print("Dataset saved as houseprice_cleaned.csv")
```

```
Dataset saved as houseprice_cleaned.csv
```

Dataset 3: Healthcare Patient Dataset

1. Load the Healthcare dataset using Pandas.

```
import pandas as pd

df = pd.read_csv("Healthcare_Patient_Dataset.csv")

print("Healthcare Dataset:")
display(df)
```

*** Healthcare Dataset:

	Patient_ID	Age	Gender	Height	Weight	Blood_Pressure	Blood_Sugar	Diagnosis
0	P001	25	Male	172	68.0	118.0	92.0	Healthy
1	P002	42	Female	160	74.0	135.0	145.0	Diabetes
2	P003	36	Male	178	82.0	128.0	98.0	Healthy
3	P004	58	Female	155	69.0	150.0	180.0	Hypertension
4	P005	47	Male	169	85.0	142.0	165.0	Diabetes
5	P006	31	Female	162	58.0	120.0	90.0	Healthy
6	P007	65	Male	170	92.0	165.0	210.0	Hypertension
7	P008	29	Female	158	54.0	115.0	88.0	Healthy
8	P009	54	Male	175	87.0	148.0	172.0	Diabetes
9	P010	39	Female	164	63.0	130.0	105.0	Healthy
10	P011	45	Male	171	79.0	138.0	150.0	Diabetes
11	P012	52	Female	159	71.0	145.0	168.0	Hypertension
12	P013	34	Male	176	76.0	126.0	96.0	Healthy
13	P014	61	Female	157	66.0	155.0	185.0	Hypertension
14	P015	49	Male	173	88.0	140.0	160.0	Diabetes
15	P016	27	Female	161	55.0	118.0	89.0	Healthy
16	P017	56	Male	174	90.0	152.0	178.0	Hypertension
17	P018	38	Female	163	61.0	128.0	102.0	Healthy
18	P019	44	Male	168	80.0	136.0	148.0	Diabetes
19	P020	33	Female	160	57.0	122.0	94.0	Healthy

URK25CS7091

2. Examine the dataset structure and identify missing values in each attribute.

```
print("Dataset Information:")
df.info()

print("\nMissing Values in Each Attribute:")
print(df.isnull().sum())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43 entries, 0 to 42
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Patient_ID      43 non-null    object
1   Age             43 non-null    int64
2   Gender          42 non-null    object
3   Height          43 non-null    int64
4   Weight          42 non-null    float64
5   Blood_Pressure  42 non-null    float64
6   Blood_Sugar     42 non-null    float64
7   Diagnosis       43 non-null    object
dtypes: float64(3), int64(2), object(3)
memory usage: 2.8+ KB
```

```
Missing Values in Each Attribute:
Patient_ID      0
Age             0
Gender          1
Height          0
Weight          1
Blood_Pressure  1
Blood_Sugar     1
Diagnosis       0
dtype: int64
```

URK25CS7091

3. Replace missing numerical values using median imputation and categorical values using mode.

```
import numpy as np
numerical_columns = df.select_dtypes(include=np.number).columns
for column in numerical_columns:
    df[column] = df[column].fillna(df[column].median())
categorical_columns = df.select_dtypes(include="object").columns
for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])

print("Missing values after imputation:")
print(df.isnull().sum())
```

```
Missing values after imputation:
Patient_ID      0
Age             0
Gender          0
Height          0
Weight          0
Blood_Pressure  0
Blood_Sugar     0
Diagnosis       0
dtype: int64
```

URK25CS7091

4. Encode the categorical variables **Gender** and **Diagnosis** using Label Encoding.

```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
df["Diagnosis"] = le.fit_transform(df["Diagnosis"])
print("Dataset after Label Encoding:")
display(df)

```

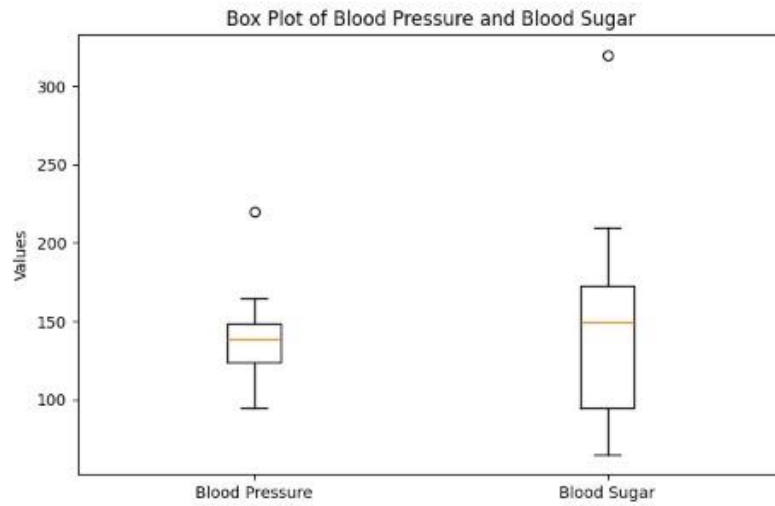
Dataset after Label Encoding:

	Patient_ID	Age	Gender	Height	Weight	Blood_Pressure	Blood_Sugar	Diagnosis
0	P001	25	1	172	68.0	118.0	92.0	1
1	P002	42	0	160	74.0	135.0	145.0	0
2	P003	36	1	178	82.0	128.0	98.0	1
3	P004	58	0	155	69.0	150.0	180.0	2
4	P005	47	1	169	85.0	142.0	165.0	0
5	P006	31	0	162	58.0	120.0	90.0	1
6	P007	65	1	170	92.0	165.0	210.0	2
7	P008	29	0	158	54.0	115.0	88.0	1
8	P009	54	1	175	87.0	148.0	172.0	0
9	P010	39	0	164	63.0	130.0	105.0	1
10	P011	45	1	171	79.0	138.0	150.0	0
11	P012	52	0	159	71.0	145.0	168.0	2
12	P013	34	1	176	76.0	126.0	96.0	1
13	P014	61	0	157	66.0	155.0	185.0	2
14	P015	49	1	173	88.0	140.0	160.0	0
15	P016	27	0	161	55.0	118.0	89.0	1
16	P017	56	1	174	90.0	152.0	178.0	2
17	P018	38	0	163	61.0	128.0	102.0	1

URK25CS7091

5. Detect abnormal values in **Blood Pressure** and **Blood Sugar** using box plots.

..



```
Blood_Pressure Outliers:  
Patient_ID Blood_Pressure  
41      P041      220.0
```

```
Blood_Sugar Outliers:  
Patient_ID Blood_Sugar  
41      P041      320.0
```

URK25CS7091

6. Normalize the numerical health indicators using Min-Max Scaling.

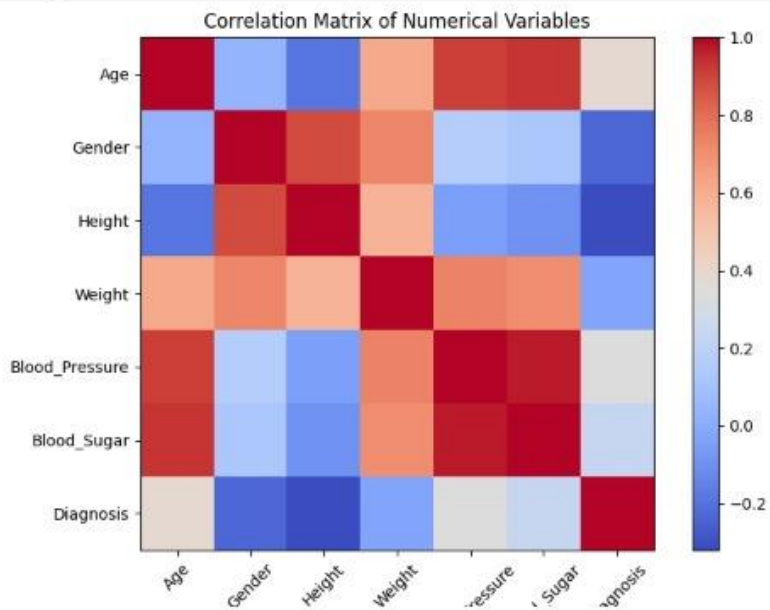
Dataset after Min-Max Scaling:

	Patient_ID	Age	Gender	Height	Weight	Blood_Pressure	Blood_Sugar	Diagnosis	
0	P001	0.062500	1	0.666667	0.356164	0.184	0.105882	1	
1	P002	0.416667	0	0.222222	0.438356	0.320	0.313725	0	
2	P003	0.291667	1	0.888889	0.547945	0.264	0.129412	1	
3	P004	0.750000	0	0.037037	0.369863	0.440	0.450980	2	
4	P005	0.520833	1	0.555556	0.589041	0.376	0.392157	0	
5	P006	0.187500	0	0.296296	0.219178	0.200	0.098039	1	
6	P007	0.895833	1	0.592593	0.684932	0.560	0.568627	2	
7	P008	0.145833	0	0.148148	0.164384	0.160	0.090196	1	
8	P009	0.666667	1	0.777778	0.616438	0.424	0.419608	0	
9	P010	0.354167	0	0.370370	0.287671	0.280	0.156863	1	
10	P011	0.479167	1	0.629630	0.506849	0.344	0.333333	0	
11	P012	0.625000	0	0.185185	0.397260	0.400	0.403922	2	
12	P013	0.250000	1	0.814815	0.465753	0.248	0.121569	1	
13	P014	0.812500	0	0.111111	0.328767	0.480	0.470588	2	
14	P015	0.562500	1	0.703704	0.630137	0.360	0.372549	0	
15	P016	0.104167	0	0.259259	0.178082	0.184	0.094118	1	
16	P017	0.708333	1	0.740741	0.657534	0.456	0.443137	2	
17	P018	0.333333	0	0.333333	0.260274	0.264	0.145098	1	
18	P019	0.458333	1	0.518519	0.520548	0.328	0.325490	0	
19	P020	0.229167	0	0.222222	0.205479	0.216	0.113725	1	URK25CS7891

7. Create a correlation matrix of the numerical variables after preprocessing.

Correlation Matrix:

	Age	Gender	Height	Weight	Blood_Pressure	Blood_Sugar	Diagnosis
Age	1.000000	0.036502	-0.187900	0.609872	0.909324	0.928787	0.376800
Gender	0.036502	1.000000	0.885913	0.722732	0.161098	0.124343	-0.236083
Height	-0.187900	0.885913	1.000000	0.575172	-0.047688	-0.095353	-0.319282
Weight	0.609872	0.722732	0.575172	1.000000	0.738264	0.702445	-0.030383
Blood_Pressure	0.909324	0.161098	-0.047688	0.738264	1.000000	0.969827	0.338999
Blood_Sugar	0.928787	0.124343	-0.095353	0.702445	0.969827	1.000000	0.239569
Diagnosis	0.376800	-0.236083	-0.319282	-0.030383	0.338999	0.239569	1.000000



URK25CS7091

8. Save the processed dataset as **healthcare_processed.csv**.

```
# Save the processed dataset
df.to_csv("healthcare_processed.csv", index=False)

print("Processed dataset saved successfully as healthcare_processed.csv")

Processed dataset saved successfully as healthcare_processed.csv
```

Dataset 4: Online Retail Sales Dataset

1. Import the Online Retail Sales dataset into Python using Pandas.

```
import pandas as pd

df = pd.read_csv("onlinetail.csv", compression="gzip", encoding="latin1")

print(df)
```

```
URK25CS7091
```

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	
...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	

	InvoiceDate	UnitPrice	CustomerID	Country
0	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	12/1/2010 8:26	3.39	17850.0	United Kingdom
...	...	...	...	...
541904	12/9/2011 12:50	0.85	12680.0	France
541905	12/9/2011 12:50	2.10	12680.0	France
541906	12/9/2011 12:50	4.15	12680.0	France
541907	12/9/2011 12:50	4.15	12680.0	France
541908	12/9/2011 12:50	4.95	12680.0	France

[541909 rows x 8 columns]

2. Identify duplicate invoices and remove duplicate records from the dataset.

```
print("Duplicate records:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates after removal:", df.duplicated().sum())
```

```
URK25CS7091
```

```
Duplicate records: 5268
Duplicates after removal: 0
```

3. Detect missing values in **Customer_ID** and **InvoiceDate**, and apply suitable preprocessing techniques.

```

print("Missing values before:")
print(df[['CustomerID', 'InvoiceDate']].isnull().sum())

df['CustomerID'] = df['CustomerID'].fillna(df['CustomerID'].median())

df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

print("\nMissing values after:")
print(df[['CustomerID', 'InvoiceDate']].isnull().sum())

```

URK25CS7091

```

Missing values before:
CustomerID    135037
InvoiceDate         0
dtype: int64

```

```

Missing values after:
CustomerID         0
InvoiceDate         0
dtype: int64

```

4. Encode the **Country** attribute using One-Hot Encoding.

```
df = pd.get_dummies(df, columns=['Country'], dtype=int)
```

```
print(df.head())
```

URK25CS7091

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	

	InvoiceDate	UnitPrice	CustomerID	Country_Australia	\
0	2010-12-01 08:26:00	2.55	17850.0	0	
1	2010-12-01 08:26:00	3.39	17850.0	0	
2	2010-12-01 08:26:00	2.75	17850.0	0	
3	2010-12-01 08:26:00	3.39	17850.0	0	
4	2010-12-01 08:26:00	3.39	17850.0	0	

	Country_Austria	Country_Bahrain	...	Country_RSA	Country_Saudi Arabia	\
0	0	0	...	0	0	
1	0	0	...	0	0	
2	0	0	...	0	0	
3	0	0	...	0	0	
4	0	0	...	0	0	

	Country_Singapore	Country_Spain	Country_Sweden	Country_Switzerland	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Country_USA	Country_United Arab Emirates	Country_United Kingdom	\
0	0	0	1	
1	0	0	1	
2	0	0	1	
3	0	0	1	
4	0	0	1	

	Country_Unspecified
0	0
1	0
2	0
3	0
4	0

[5 rows x 45 columns]

5. Detect outliers in **Quantity** and **UnitPrice** using the IQR method.

```

for column in ['Quantity', 'UnitPrice']:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    print(column, "Outliers:")
    print(df[(df[column] < lower) | (df[column] > upper)].shape[0])

    df = df[(df[column] >= lower) & (df[column] <= upper)]

print(df)

```

URK25C57091

```

38301
UnitPrice Outliers:
32348

```

	InvoiceNo	StockCode	Description	Quantity
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6
1	536365	71053	WHITE METAL LANTERN	6
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6
...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3

	InvoiceDate	UnitPrice	CustomerID	Country_Australia
0	2010-12-01 08:26:00	2.55	17850.0	0
1	2010-12-01 08:26:00	3.39	17850.0	0
2	2010-12-01 08:26:00	2.75	17850.0	0
3	2010-12-01 08:26:00	3.39	17850.0	0
4	2010-12-01 08:26:00	3.39	17850.0	0
...	...	...	...	...
541904	2011-12-09 12:50:00	0.85	12680.0	0
541905	2011-12-09 12:50:00	2.10	12680.0	0
541906	2011-12-09 12:50:00	4.15	12680.0	0
541907	2011-12-09 12:50:00	4.15	12680.0	0
541908	2011-12-09 12:50:00	4.95	12680.0	0

6. Normalize the **Quantity** and **Unit_Price**, attributes using Min-Max Scaling.

```

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['Quantity', 'UnitPrice']] = scaler.fit_transform(
    df[['Quantity', 'UnitPrice']]
)

print(df[['Quantity', 'UnitPrice']].head())

```

```

    Quantity  UnitPrice
0  0.514286   0.295824
1  0.514286   0.393271
2  0.571429   0.319026
3  0.514286   0.393271
4  0.514286   0.393271

```

7. Generate summary statistics before and after preprocessing to evaluate data quality improvements.

```

before = df.describe()
print("Before Preprocessing:")
print(before)

```

```

print("\nAfter Preprocessing:")
print(df.describe())

```

```

    min      0.000000      2010-12-01 08:26:00      0.000000
    25%      0.371429      2011-03-27 14:28:00      0.145012
    50%      0.428571      2011-07-20 11:38:00      0.243619
    75%      0.571429      2011-10-19 16:59:00      0.435035
    max      1.000000      2011-12-09 12:50:00      1.000000
    std      0.131750                                NaN      0.237004

```

```

    count      CustomerID  Country_Australia  Country_Austria  Country_Bahrain \
    mean      15286.197487          0.001214          0.000722          0.000022
    min      12347.000000          0.000000          0.000000          0.000000
    25%      14465.000000          0.000000          0.000000          0.000000
    50%      15145.000000          0.000000          0.000000          0.000000
    75%      16283.000000          0.000000          0.000000          0.000000
    max      18287.000000          1.000000          1.000000          1.000000
    std      1465.643583          0.034815          0.026866          0.004736

```

```

    count      Country_Belgium  Country_Brazil  Country_Canada  ...  Country_RSA \
    mean          0.003576          0.000052          0.000238  ...          0.000114
    min          0.000000          0.000000          0.000000  ...          0.000000
    25%          0.000000          0.000000          0.000000  ...          0.000000
    50%          0.000000          0.000000          0.000000  ...          0.000000
    75%          0.000000          0.000000          0.000000  ...          0.000000
    max          1.000000          1.000000          1.000000  ...          1.000000
    std          0.059690          0.007183          0.015418  ...          0.010695

```

	Country_Saudi Arabia	Country_Singapore	Country_Spain	Country_Sweden \
count	445792.000000	445792.000000	445792.000000	445792.000000
mean	0.000022	0.000285	0.004345	0.000455
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000
std	0.004736	0.016876	0.065774	0.021335

	Country_Switzerland	Country_USA	Country_United Arab Emirates \
count	445792.000000	445792.000000	445792.000000
mean	0.003152	0.000485	0.000110
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000
std	0.056051	0.022007	0.010484

	Country_United Kingdom	Country_Unspecified
count	445792.000000	445792.000000
mean	0.927208	0.000823
min	0.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	0.000000
75%	1.000000	0.000000
max	1.000000	1.000000
std	0.259795	0.028681

8. Save the cleaned dataset as **retail_sales_cleaned.csv**.

```
df.to_csv("retail_sales_cleaned.csv", index=False)
```

```
print("Dataset saved as retail_sales_cleaned.csv")
```

```
Dataset saved as retail_sales_cleaned.csv
```